

Assignment6

Info

Name: Cody (Zhexian) Liu

Assignment Number: 6

Repo Link: <https://github.com/Rorical/CWM-FDI>

Code Link: <https://github.com/Rorical/CWM-FDI/blob/main/assignment6/main.py>

Project Information

Overview

A command-line tool that estimates the transport-layer energy consumption and carbon footprint of a network request. Given a target domain or IP, the script runs traceroute, resolves each hop to its country and carbon intensity via the Green Web Foundation API, and applies a transport-layer adaptation of the Sustainable Web Design v4 (SWDM v4) model, using SWDM v4 network intensities and a separately stated heuristic for datacentre networking energy.

Run Log

```
(assignment6) ubuntu@ubuntu:~/CWM-FDI/assignment6$ /usr/bin/python
/home/ubuntu/CWM-FDI/assignment6/main.py
Transport energy estimate for 1.0 GB request to google.com
```

```
Route: 14 visible + 6 silent = 20 hops
Network energy:      0.072000 kWh
DC networking:       0.016750 kWh
Total transport:     0.088750 kWh
Network carbon:      18.012 gCO2
DC networking carb:  6.342 gCO2
Total transport CO2: 24.354 gCO2
```

Per-hop breakdown:

```
163.1.26.134      World      0.003600 kWh  ×
442 gCO2/kWh = 1.592028 gCO2
193.63.109.41     World      0.003600 kWh  ×
442 gCO2/kWh = 1.592028 gCO2
193.63.108.69     United Kingdom 0.003600 kWh  ×
268 gCO2/kWh = 0.965718 gCO2
146.97.37.193     United Kingdom 0.003600 kWh  ×
268 gCO2/kWh = 0.965718 gCO2
146.97.33.6       United Kingdom 0.003600 kWh  ×
```

```

268 gCO2/kWh = 0.965718 gCO2
    146.97.33.61      United Kingdom      0.003600 kWh ×
268 gCO2/kWh = 0.965718 gCO2
    146.97.35.18      United Kingdom      0.003600 kWh ×
268 gCO2/kWh = 0.965718 gCO2
    72.14.205.74      World      0.003600 kWh ×
442 gCO2/kWh = 1.592028 gCO2
    192.178.97.117    World      0.003600 kWh ×
442 gCO2/kWh = 1.592028 gCO2
    192.178.97.191    United States of America 0.003600 kWh ×
379 gCO2/kWh = 1.363050 gCO2
    192.178.98.3      United States of America 0.003600 kWh ×
379 gCO2/kWh = 1.363050 gCO2
    142.250.239.28     United States of America 0.003600 kWh ×
379 gCO2/kWh = 1.363050 gCO2
    192.178.252.204    United States of America 0.003600 kWh ×
379 gCO2/kWh = 1.363050 gCO2
    142.250.129.101    United States of America 0.003600 kWh ×
379 gCO2/kWh = 1.363050 gCO2

```

Design

The tool is structured as a single Python script with four logical stages:

1. **Route discovery:** executes `traceroute <target>` and parses the output using regex. Each line is decomposed into hop number, hostname, IP, and RTT per probe. Silent hops and multiple IPs per hops are handled properly using world average and first selection estimation.
2. **Private IP resolution:** private IPs behind NAT are mapped to their nearest public IP by probe position. The mapping is bidirectional: if a public IP appears both before and after a private IP at equal distance, the upstream earlier public wins since that is the likely NAT gateway. This provides a roughly good estimation for the location of private routers.
3. **Geo & carbon lookup:** each unique public IP is queried against `api.thegreenwebfoundation.org/api/v3/ip-to-co2intensity/{ip}`. The API returns country name, carbon intensity (gCO_2/kWh), and data year. Results are cached in memory so no redundant requests are made.
4. **Energy estimation:** applies a transport-layer adaptation of the [SWDM v4](#) model, using SWDM v4 network intensity values and a separately stated heuristic for datacentre networking energy:
 - Network energy: $E_{\text{net}} = D \times (0.059 + 0.013) \text{ kWh}$
 - DC networking: $E_{\text{dc}} = D \times (0.055 + 0.012) \times 0.25 \text{ kWh}$
 - Per-hop energy: $E_{\text{hop}} = (E_{\text{net}} + E_{\text{dc}})/H$

- Per-hop carbon: $C_{\text{hop}} = E_{\text{hop}} \times I_{\text{hop}}$

where D = data volume in GB, H = total hops (visible + silent), I_{hop} = carbon intensity in gCO_2/kWh at that hop.

Data sources, assumptions and caveats

Data sources:

- [Green Web Foundation](https://api.thegreenwebfoundation.org/api/v3/ip-to-co2intensity/{ip}) IP-to-CO2 API (`api.thegreenwebfoundation.org/api/v3/ip-to-co2intensity/{ip}`): provides country name, carbon intensity (gCO_2/kWh) and data year per public IP. Returns global average ($442 gCO_2/kWh$, "World") for IPs not in their dataset.
- [SWDM v4](#) intensities: network 0.059 (operational) + 0.013 (embodied) kWh/GB; datacentre 0.055 + 0.012 kWh/GB. DC networking is approximated as 25 % of datacentre energy as a heuristic assumption; this value is not directly measured for the target datacentre and should be tested in sensitivity analysis.

Operational emissions

To estimate operational emissions, first, we define the energy consumption intensity. To do so, we follow a top-down approach, which means that we look at the global energy consumption for each segment, as well as total data transfer. Here are the values we use for our calculations:

- Total energy consumption for data centers: 290 TWh ([IEA 2022](#))
- Total energy consumption for the network: 310 TWh ([IEA 2022](#))
- Total energy consumption for user devices: 421 TWh ([Malmodin 2023](#))
- Total data transfer across the internet: 5.29 ZB ([ITU 2023](#))

By dividing the energy consumption by the total data transfer, we get the energy intensity based on data transfer for each segment:

$$\text{Energy intensity (kWh/GB)} = \text{Energy consumption (kWh)} / \text{Data transfer (GB)}$$

Then, to get the estimated energy use for each segment, we perform the following calculations (converting TWh to kWh and ZB to GB as we go):

$$\text{Operational energy intensity data centers} = (290 \times 10^9 \text{ (kWh)}) / (5.29 \times 10^{12} \text{ (GB)}) = 0.055 \text{ kWh/GB}$$

$$\text{Operational energy intensity networks} = (310 \times 10^9 \text{ (kWh)}) / (5.29 \times 10^{12} \text{ (GB)}) = 0.059 \text{ kWh/GB}$$

$$\text{Operational energy intensity user devices} = (421 \times 10^9 \text{ (kWh)}) / (5.29 \times 10^{12} \text{ (GB)}) = 0.080 \text{ kWh/GB}$$

The final values we obtain for operational energy intensity are:

- **Data centers:** 0.055 kWh/GB
- **Network:** 0.059 kWh/GB
- **User devices:** 0.080 kWh/GB

Once the energy intensity is known, we can use the carbon intensity previously mentioned to obtain the carbon intensity of data transfer for each of the segments.

$$\text{Operational emissions for segment (gCO}_2\text{e/GB)} = \text{Data transfer (GB)} \times \text{Energy intensity (kWh/GB)} \times \text{Grid carbon intensity (gCO}_2\text{e/kWh)}$$

Once the energy intensity is known, we can use the carbon intensity previously mentioned to obtain the carbon intensity of data transfer for each of the segments.

Operational emissions for segment (gCO₂e/GB) = Data transfer (GB) × Energy intensity (kWh/GB) × Grid carbon intensity (gCO₂e/kWh)

You can estimate this for each segment, and then sum the totals to get the total estimated emissions. In the example below, we use a global average grid intensity of 494 grams CO₂e/kWh which is pulled from the CO₂ intensity dataset for “World” of Ember’s Data Explorer. This can be replaced by numbers for the specific country or state where this is known ([see FAQ](#)).

Operational emissions (gCO₂e/GB) = OP_{DC} + OP_N + OP_{UD}

Where:

Operational emissions data centers (OP_{DC}) = Data transferred (GB) × 0.055 kWh/GB × 494 gCO₂e/kWh

Operational emissions networks (OP_N) = Data transferred (GB) × 0.059 kWh/GB × 494 gCO₂e/kWh

Operational emissions user devices (OP_{UD}) = Data transferred (GB) × 0.080 kWh/GB × 494 gCO₂e/kWh

Embodied emissions

To estimate embodied emissions, we assume that the values provided by Malmodin are equivalent to electricity consumption with a global electricity emissions factor (421 TWh, [Malmodin 2023](#)). For calculating embodied emissions, we have allocated a portion of the total LCA energy consumption to each of the three system segments based on [research from Jens Malmodin and co authors in 2023](#). The values we use are the following:

- Percentage of total embodied energy for data centers: 11% (62 TWh)
- Percentage of total embodied energy for the network: 12% (68 TWh)
- Percentage of total embodied energy for user devices: 77% (430 TWh)

We can then apply the same process as before to estimate embodied energy and plug that into an embodied GHG estimation calculation for each segment.

Embodied energy intensity data centers = $(62 \times 10^9 \text{ (kWh)}) / (5.29 \times 10^{12} \text{ (GB)}) = 0.012 \text{ kWh/GB}$

Embodied energy intensity networks = $(68 \times 10^9 \text{ (kWh)}) / (5.29 \times 10^{12} \text{ (GB)}) = 0.013 \text{ kWh/GB}$

Embodied energy intensity user devices = $(430 \times 10^9 \text{ (kWh)}) / (5.29 \times 10^{12} \text{ (GB)}) = 0.081 \text{ kWh/GB}$

The final values we obtain for embodied emissions are:

- **Data centers:** 0.012 kWh/GB
- **Network:** 0.013 kWh/GB
- **User devices:** 0.081 kWh/GB

Assumptions:

- Symmetric routing: return path follows the same route and consumes the same energy.
- Uniform per-hop energy: all hops receive an equal share of total transport energy, since per-router power profiles are unavailable from traceroute.

- Silent hops still forward traffic even though they do not respond to traceroute probes.
- Annual average carbon intensities from Ember (from the [Green Web Foundation](#)) are used; real-time marginal intensities would differ.
- Client-device processing and server request processing are excluded.

Caveats:

- The [Green Web Foundation](#) dataset has limited coverage; many IPs fall back to "World" (global average, 442 gCO₂/kWh from 2021), which is a coarse approximation.
- Private IPs under NAT inherit the geo and carbon data of their nearest public IP — this mapping depends on the correctness of the NAT topology assumption.
- Load-balanced multi-path hops (multiple IPs per traceroute line) list all responding routers but only the first unique public IP per hop is used for the hop count; variability in path selection across probes is not modelled.
- Traceroute captures a single snapshot of the route; actual paths may vary due to BGP changes, ECMP, or anycast.
- The 25 % DC-network attribution heuristic is a rough industry guideline, not a measured value for any specific datacentre.

Relationship to CWM-FDI

This project builds directly on concepts from Assignment 3 (Power, Energy & Carbon):

- **Carbon footprint formula** ($CFP = E \times CI$): used in Assignment 3 Question 13 to compute matmul carbon footprints at different locations. This project applies the same formula per-hop, replacing the fixed CI with per-location values from the Green Web Foundation API.
- **Per-hop network energy**: asked what the network energy impact is when data travels through multiple route. This project answers that question quantitatively by summing SWDM v4 energy across every visible and silent hop.
- **Theoretical vs. measured energy**: Assignment 3 compared a theoretical NIC-only estimate (2.4–8 J for 1 GB) against a measured system-level value (393.71 J). Our project uses the SWDM v4 network-level intensities (0.072 kWh/GB $\approx$ 259 J/GB for networking alone). This is coarser than NIC-only but more realistic than system-level for transport-layer estimation.
- **Carbon intensity sources**: Assignment 3 used NESO UK and Electricity Maps for fixed-location CI lookups. This project replaces manual CI selection with automatic per-hop API lookups via the Green Web Foundation, providing more accurate estimations.

Use of AI declaration

This project uses generative AI in two places:

1. LLM is being used to generate Regex expression for parsing the console log of traceroute tool. This part is tedious and requires careful table looking.
 2. LLM is being used to assist the algorithm used to find public IP address related to a private IP. I provide the idea: Assuming private IPs are under NATs, then their corresponding public IP must be the nearest public IP around it on route hops.
- Model: DeepSeek V4 Flash
 - Variant: Max, 1M context

Logs

```
traceroute google.com
traceroute to google.com (142.250.117.138), 30 hops max, 60 byte packets
 1 thom-net-wrd-gw-int (10.136.7.254)  0.851 ms  0.809 ms  0.741 ms
 2 163.1.26.134 (163.1.26.134)  0.962 ms  1.270 ms  1.724 ms
 3 proq-frodo-050133.frodo.ox.ac.uk (172.24.17.134)  2.238 ms pmus-frodo-
050133.frodo.ox.ac.uk (172.24.65.134)  0.805 ms  0.755 ms
 4 groq2-pmus.odin.ox.ac.uk (172.31.4.242)  2.120 ms  0.713 ms  1.038 ms
 5 ae11-786.harwat-rbr1.ja.net (193.63.109.41)  0.930 ms  0.939 ms  0.892
ms
 6 ae7.readdy-rbr1.ja.net (193.63.108.69)  1.897 ms  1.843 ms  1.887 ms
 7 ae19.londpg-sbr2.ja.net (146.97.37.193)  2.832 ms  2.787 ms  2.758 ms
 8 ae30.londtw-sbr2.ja.net (146.97.33.6)  3.244 ms  3.276 ms  3.313 ms
 9 ae28.londtt-sbr1.ja.net (146.97.33.61)  6.870 ms  6.798 ms  6.833 ms
10 ae25.londtt-ban1.ja.net (146.97.35.18)  3.291 ms  3.325 ms  3.451 ms
11 72.14.205.74 (72.14.205.74)  3.342 ms  3.236 ms  3.200 ms
12 192.178.97.117 (192.178.97.117)  5.978 ms 192.178.97.51 (192.178.97.51)
5.336 ms 192.178.98.3 (192.178.98.3)  3.909 ms
13 192.178.254.100 (192.178.254.100)  3.657 ms 216.239.46.88
(216.239.46.88)  4.977 ms 192.178.87.72 (192.178.87.72)  4.038 ms
14 * * *
15 * * *
16 * * *
17 * * *
18 * * *
19 * * *
20 * * *
21 um-in-f138.1e100.net (142.250.117.138)  4.074 ms  3.529 ms  4.073 ms
```

Write two regex expressions to parse the content of traceroute.

requirements: 1. one line to identify the hops, and one line to identify the results inside each hop. 2. Must be in python's expression.

Here are the two regex expressions to parse the traceroute output, as requested:

```
import re
# Identify the hops
re.compile(r'\s*(\d+)\s+(.*)')

# Identify the results inside each hop
re.compile(
r'\*|'
r'(\S+) \((\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})\) | '
r'(\d+\.\d+)\s*ms',
)
```

```
traceroute 1.1.1.1
traceroute to 1.1.1.1 (1.1.1.1), 30 hops max, 60 byte packets
 1 thom-net-wrd-gw-int (10.136.7.254) 0.286 ms 0.269 ms 0.253 ms
 2 163.1.26.134 (163.1.26.134) 1.211 ms 0.875 ms 0.857 ms
 3 proq-frodo-050133.frodo.ox.ac.uk (172.24.17.134) 1.205 ms 1.339 ms
0.889 ms
 4 groq2-pmus.odin.ox.ac.uk (172.31.4.242) 1.322 ms 1.292 ms 0.981 ms
 5 ae11-786.harwat-rbr1.ja.net (193.63.109.41) 0.879 ms 0.874 ms 0.897
ms
 6 ae7.readdy-rbr1.ja.net (193.63.108.69) 1.827 ms 1.810 ms 1.817 ms
 7 ae19.londpg-sbr2.ja.net (146.97.37.193) 2.852 ms 2.812 ms 2.817 ms
 8 ae30.londtw-sbr2.ja.net (146.97.33.6) 3.635 ms 3.573 ms 3.369 ms
 9 ae28.londtt-sbr1.ja.net (146.97.33.61) 4.765 ms 5.217 ms 5.031 ms
10 ae25.londtt-ban1.ja.net (146.97.35.18) 3.532 ms 3.900 ms 3.512 ms
11 141.101.71.14 (141.101.71.14) 3.388 ms 3.431 ms 3.475 ms
12 141.101.71.1 (141.101.71.1) 3.586 ms 3.436 ms 3.402 ms
13 141.101.71.5 (141.101.71.5) 3.673 ms 3.487 ms 3.642 ms
14 141.101.71.107 (141.101.71.107) 4.824 ms 4.261 ms 3.897 ms
15 141.101.71.61 (141.101.71.61) 4.183 ms 4.263 ms 4.250 ms
16 1.1.1.1 (1.1.1.1) 3.278 ms 3.194 ms 3.149 ms
```

The current private-to-public IP mapping uses the last seen public IP. Shouldn't a target-side private IP map to the nearest public IP in either direction rather than always the client-side gateway?

Good point. The current algorithm only looks backward to `last\_public`, which would misattribute a target-side private IP to the client-side NAT gateway. A better approach is to find the nearest public IP by probe position in either direction, with ties favouring the upstream (earlier) public since that is the likely NAT gateway. I will rewrite `\_build\_ip\_inheritance` to use this bidirectional nearest-neighbour approach.

```
def _build_ip_inheritance(hops: list[dict]) -> list[str]:
    all_ips = [p['ip'] for h in hops for p in h['probes'] if p['ip']]

    public_positions: list[tuple[int, str]] = [
        (i, ip) for i, ip in enumerate(all_ips) if _is_public_ip(ip)
    ]

    ordered_publics: list[str] = []
    for _, ip in public_positions:
        if ip not in ordered_publics:
            ordered_publics.append(ip)
        if ip not in _gw_cache:
            _gw_cache[ip] = None

    first_public = public_positions[0][1] if public_positions else None
    if first_public is None:
        return ordered_publics

    for i, ip in enumerate(all_ips):
        if _is_public_ip(ip) or ip in _ip_to_public:
            continue
        best = first_public
        best_dist = float('inf')
        for pos, pub_ip in public_positions:
            dist = abs(i - pos)
            if dist < best_dist or (dist == best_dist and pos < i):
                best_dist = dist
                best = pub_ip
        _ip_to_public[ip] = best

    return ordered_publics
```

This ensures target-side private IPs (e.g. behind a destination NAT) inherit from their nearest public gateway rather than the client-side one.

